Part 1 Onboarding Guide

Storefront | POS Project (video game store)

Written for Ben. Prepared Oct 20, 2026 (Day 5 of the plan). Based on the Part Briefs (Oct 16), the current schema design document (updated Oct 16, evening) and the project plan.

1. At a glance

Item	Detail
Your part	Part 1: Storefront. Everything a customer sees before paying, plus their account and notifications. Part 2 (checkout) reads the cart you build, through the view `VW_CART`.
Branch	`part-1-storefront`. Pull requests go to the lead, who merges to main. Vercel deploys from main only.
Folders you own	`db/storefront/`, `api/products/`, `api/cart/`, `api/reviews/`, `api/account/`, `api/jobs/`, `src/pages/storefront/`.
You build	1 trigger (`TR_REVIEW_Guard`), the catalog, product, cart and account queries, 4 API areas (products, reviews, cart, account) plus the notification job, 5 screens (catalog, product page, cart, account, notifications), SQL tests, and a Jest test per endpoint.
Already passed	Foundation ready (Oct 17), schema freeze (Oct 18) and the walking skeleton (Oct 19: log in, browse, add to cart, pay, see the order, live on Vercel). Browse and add to cart are your part of that skeleton, so check with the lead what is already in main.
Still ahead	**Tomorrow, Oct 21 (Day 6):** your `triggers.sql` and `test.sql` are due to the lead. Oct 22 (Day 7): last build day, and the full chain should run through the app. Oct 23: **feature freeze**. Oct 23 to 24: bug bash. Oct 25: submission.
Priority	No never-cut item is yours, but catalog and cart feed the walking skeleton and Part 2. If you fall behind, tell the lead early (cut list: section 8).

2. Documents to read first

Document	How to use it
schema-current-design.md	**Source of truth** for tables, columns, triggers, decisions and rebuild steps, newer than the conceptual schema PDF. It holds the full text of the 5 built triggers.
POS Project: Part Briefs (PDF)	Your Part 1 page plus the shared rules, the lead's deliverables, the timeline and the cut order.
Conceptual schema PDF	**Outdated. Ignore it for design.** It still has Register, Cashier, nullable Customer_id on transactions and cash payments, which were all removed.
Technology resources PDF	Tutorials for SQL Server, Node.js and React. Useful for learning, not for decisions.
triggers.sql, test_triggers.sql, drop.sql, schema file, seed script	**Not attached to this guide. Ask the lead for them** (section 4). `triggers.sql` is the style model for `TR_REVIEW_Guard`, and `test_triggers.sql` is the model for your `db/storefront/test.sql`.

3. Install the tools

These steps assume Windows (SQL Server Express runs on Windows). Run commands in PowerShell or the VS Code terminal (Ctrl+`). You already have VS Code, a SQL Server Express connection and access to the team repository, so those are skipped. Git, Node.js, the project packages and the Vercel CLI are not installed yet.

Step 1: Git

1. Download Git for Windows from https://git-scm.com/download/win and run the installer with the default options.
2. Close and reopen VS Code, then run `git --version`.
3. Set your identity (use your GitHub email):

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Step 2: Node.js (LTS)

1. Download the **LTS** version from https://nodejs.org (not Current) and install with the defaults.
2. Reopen VS Code and check `node -v` and `npm -v`. Both must print a version.
3. If PowerShell says scripts are disabled when you run npm, run this once:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

Step 3: Get the project and install its packages

> **Important.** Do **not** run `npm create vite` or start a new project. The lead owns the skeleton (Vite, React Router, lib/, src/shared/). You work inside the shared repo.

1. The team repository already exists and you have access. Clone it (use the repository URL from GitHub, under the green Code button) and move into the folder.
2. Create your branch.
3. Install everything listed in the lead's `package.json`.
4. Check the stack is present.

```
cd C:\Projects
git clone <repo-url>
cd <repo-folder>
git checkout -b part-1-storefront
npm install
npm ls mssql bcrypt jsonwebtoken jest react-router-dom vite
```

- If a package is missing, **tell the lead** instead of committing a change to `package.json`, so the five parts do not conflict. To keep testing locally in the meantime: `npm install mssql bcrypt jsonwebtoken react-router-dom` and `npm install --save-dev jest vite`.

- If `bcrypt` fails to install (it is a native module), the lead can switch to `bcryptjs`. Otherwise install "Desktop development with C++" from the Visual Studio Build Tools and retry.

Step 4: Build your local database (skip if pos_store already exists)

Your VS Code connection only gives you the server. The `pos_store` database and its data come from the lead's SQL files. Check first: in VS Code run `SELECT DB_ID('pos_store');` on `master`. If it returns a number, skip to Step 5. Otherwise:

1. On `master`, run `CREATE DATABASE pos_store;`.
2. Run the schema file **once**. It starts with `USE pos_store;` and does not create the database.
3. Run `triggers.sql` **once** (it also starts with `USE pos_store;` then `GO`).
4. Run the seed script **once**. It must end with `UPDATE PRODUCT SET Stock_quantity = Stock_quantity;` so low-stock products get their alert, which needs the triggers to exist already. Ask the lead to confirm the seed script's place in this order.

- Each file runs exactly once. Running the schema file twice gives "There is already an object named ..." for every table.
- Do **not** run `test_triggers.sql` on a seeded database. The schema document says it must run on a database with no active reward program, and the seed script loads one.
- To start over on your own machine, run this from `master` (it also does step 1) and then repeat steps 2 to 4:

```
ALTER DATABASE pos_store SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
DROP DATABASE pos_store;
CREATE DATABASE pos_store;
```

Step 5: Let Node connect to your local SQL Express

VS Code's SQL extension can connect in ways Node's `mssql` package cannot, so this step is often the real blocker.

1. Open **SQL Server Configuration Manager**. Under SQL Server Network Configuration, Protocols for SQLEXPRESS, right-click **TCP/IP** and choose Enable.
2. Under SQL Server Services, set **SQL Server Browser** to Automatic and Running (needed to find the named instance).
3. Restart **SQL Server (SQLEXPRESS)** in the same list.
4. Node needs a SQL login, not Windows authentication. In VS Code, on `master`, check the mode. A result of 0 means mixed mode is already on. If it returns 1, enable "SQL Server and Windows Authentication mode" in the server Properties, Security page, then restart the service.

```
SELECT SERVERPROPERTY('IsIntegratedSecurityOnly');
```

5. Create a local development login. This is for your machine only; choose your own password.

```
CREATE LOGIN pos_dev WITH PASSWORD = 'ChooseAStrongPassword!1';
GO
USE pos_store;
GO
CREATE USER pos_dev FOR LOGIN pos_dev;
ALTER ROLE db_owner ADD MEMBER pos_dev;
GO
```

6 Put your settings in `.env.local` in the repo root. Ask the lead for the exact variable names that `lib/db.js` reads, and make sure `.env*` is in `.gitignore`. **Never commit this file.**

Setting	Local value
Server	localhost
Instance	SQLEXPRESS
Database	pos_store
User / password	The pos_dev login you created
Encryption	encrypt false and trust server certificate true (local only; the Azure VM uses encrypt: true)

Step 6: Vercel CLI (optional)

It runs the `api/` functions locally the way they run in production. The functions are plain Node handlers, not Express.

1 Install and log in: `npm install -g vercel`, then `vercel login`.

2 The lead must first add you to the Vercel project. Then run `vercel link` inside the repo.

3 Do **not** pull production environment variables unless the lead says so, because that downloads real credentials. Use your own `.env.local`.

4 Run `vercel dev` for app and API together. If the setup uses `npm run dev` for the front end only, use that for screens and Jest for the API.

Optional VS Code extensions: **ESLint** and **Prettier** if the repo uses them, and **Thunder Client** or **REST Client** to call `api/products` and `api/cart` without a screen.

Step 7: Verify the setup

[ ]	`git --version`, `node -v` and `npm -v` each print a version.
[ ]	`git branch` shows `* part-1-storefront`.
[ ]	`npm ls` shows no missing packages.
[ ]	Nested triggers are on: `SELECT value_in_use FROM sys.configurations WHERE name = 'nested triggers';` returns **1**. The triggers chain, so this is required.
[ ]	`SELECT COUNT(*) FROM sys.tables;` returns 27, and `SELECT name FROM sys.triggers;` lists at least the 5 triggers from `triggers.sql` (more once other parts merge theirs).
[ ]	`SELECT TOP 5 * FROM VW_CART;` and `SELECT COUNT(*) FROM PRODUCT;` run without error, and the second returns more than 0 after the seed script.
[ ]	A small Node script (or the lead's `query()` from `lib/db.js`, if it exists) connects as `pos_dev` and runs `SELECT 1`.

[ ]	`npx jest` runs without crashing ("no tests found" is fine), and `npm run dev` starts Vite and opens the page in the browser.

> **Troubleshooting.** If the connection fails, check TCP/IP, SQL Server Browser, mixed mode and the service restart. Send the exact error message when asking for help.

4. What you need from the lead

[ ]	Vercel project access, if you want to use the CLI (you already have access to the repository).
[ ]	`lib/db.js` (`query(sql, params)` and `withTransaction(fn)`), `lib/auth.js` (`requireAuth(['manager'])(handler)`) and `lib/http.js` (`sendJson`, `sendError`). The names were proposed and became final at the end of Day 2 (Oct 17), so use what the lead merged.
[ ]	`src/shared/` components and the API client (`api.get`, `api.post`), plus login and register, so you can test as a customer.
[ ]	The **seed script** (promised by the end of Day 2): products, customers, employees and completed sales. Helpful extras for your tests (a suggestion, ask if missing): an inactive product, active, expired and inactive discounts, a category discount, a customer with a default address and a phone, and a cart older than 24 hours.
[ ]	The schema file, `triggers.sql`, `test_triggers.sql`, `drop.sql` and the seed script as files, plus the exact variable names for `.env.local` and, when you need it, a least-privilege Azure database login (not sa).
[ ]	Agreement with Part 2 on the **VW_CART columns** (it was due on Day 2). Confirm what was agreed and that the view is in the schema file. Part 2's `sp_Checkout` reads the cart through it.
[ ]	How the notification job is started (Vercel Cron or a manual "run now" during the demo) and how its endpoint is protected. The brief says the lead decides.
[ ]	Which `THROW` error numbers are free. 50001 and 50002 are already used by built triggers.

Database rules (shared by all parts)

1. Triggers are AFTER, set-based and named `TR_<Table>_<Purpose>`. They start with `SET NOCOUNT ON; SET XACT_ABORT ON;` and fail with `THROW`. Every SQL file begins with `USE pos_store;` then `GO`.
2. Never edit `CUSTOMER.Loyalty_points` or `Tier_id` directly. Points come from a LOYALTY_TRANSACTION row, and the built trigger updates the rest. Your profile update must not touch those two columns.
3. Never recompute stock, points, tier or low-stock alerts in the app. The built triggers keep them in sync.
4. Every query is parameterized. Prices, tax and totals come from the database, never from the request.
5. A write that touches several tables runs in one database transaction, using the lead's helper.
6. Each trigger's tests go in `db/storefront/test.sql`. Every failing case runs in its own transaction and rolls back.

Rules specific to Part 1

- The app sets `Added_at` in its own UPDATE whenever a cart quantity changes, so the idle-cart reminder sees real activity. The cart page only links to Part 2's `/checkout` route; it does not create orders.
- Catalog queries return only products with `Is_active = 1`. Inactive products still show in old orders (Part 2's order detail).

- The account page has one checkbox for `Notify_cart` and one for `Notify_offers`. Build the job so that Vercel Cron or a manual "run now" both work.
- The login token carries the user id and role. Take the customer id from it for the cart, reviews and account, never from the request body (the brief's rule for employee ids, and what Part 2 does at checkout).

Decisions already closed (do not reopen)

Topic	Decision
Online only	The store is 100% online. No guest checkout: browsing and the cart for visitors live in the browser, and an account is required to purchase. CART_ITEM rows exist only for logged-in customers.
Is_active	PRODUCT and DISCOUNT have `Is_active` (default 1). Products with sales history are deactivated, not deleted. The catalog shows only active products.
Discounts	A discount applies only when `Is_active = 1` and today is within Start_date and End_date. Per line, the price uses the **single largest saving** among the product discount and the category discount. No stacking. `DISCOUNT.Code` is unused (no promo code box).
Cart	One cart per customer, no header table. Show it through `VW_CART` (joins PRODUCT, uses Sell_price). Part 2 deletes the rows at checkout after copying them.
Addresses	At most one default address per customer (a filtered unique index). Checkout copies the address into the sale's Ship_* columns, so later edits do not change old orders.
Notifications	In-site only, no email. `Notify_offers` and `Notify_cart` (default 1) let a customer turn each kind off, and the job skips them. SQL Server Express has no Agent, so a Node job does the work.
Reviews	`TR_REVIEW_Guard` is in the database (moved from the app on 2026-10-16). Only if it is cut does the app check the same rule.

> **Why this matters downstream.** Two things you build are read by other parts. Part 2's `sp_Checkout` reads your cart through `VW_CART`, so do not change the view's columns without telling Part 2. And the effective price your catalog shows must follow the same discount rule as checkout (largest of the product and category saving, active and in date), or customers will see one price in the catalog and be charged another.

6. The data you work with

Figure 1, on the last page of this guide, draws these tables and how they connect. Read the table below for the details of each column, then use the figure to see the relationships.

Table	What Part 1 needs to know
CUSTOMER	First_name, Last_name, Email (unique), Notify_offers and Notify_cart (BIT, default 1) are yours to edit. `Loyalty_points` and `Tier_id` are set by triggers.
CUSTOMER_ADDRESS	Address_id, Customer_id, Address_type, Recipient_name, Street, Apt, City, State, Zip, is_default. At most one default per customer through a filtered unique index, so when the default changes, clear the old one and set the new one in the same database transaction.
CUSTOMER_PHONE	PK (Customer_id, Phone_number). Phone management is on the cut-first list.
CART_ITEM / VW_CART	CART_ITEM has PK (Customer_id, Product_id), Quantity (> 0), Added_at. Because the key is the pair, adding a product that is already in the cart means updating its quantity. Read the cart through VW_CART.

Table	What Part 1 needs to know
PRODUCT	Product_id, SKU (unique), Name, Description, Sell_price, Stock_quantity, Product_condition ('New', 'Used', 'Refurbished'), Category_id, Discount_id (nullable), Is_active. Stock is changed by triggers, so always read it fresh.
CATEGORY / DISCOUNT	CATEGORY has Name (unique) and Discount_id (nullable). DISCOUNT has Amount, Amount_type ('percent' or 'fixed'), Start_date, End_date, Is_active. Both PRODUCT and CATEGORY can point to a discount.
REVIEW	Review_id, Rating (1 to 5), Comment, Review_date, Product_id, Customer_id, Transaction_id (nullable, ON DELETE SET NULL). A review may be linked to a purchase; `TR_REVIEW_Guard` checks that link.
NOTIFICATION	Notification_id, Customer_id, Notification_type ('cart reminder' or 'offer'), Message, Discount_id (nullable, SET NULL), Created_at, Sent_at (nullable), Read_at (nullable).
SALES_TRANSACTION / TRANSACTION_ITEM	Read only for you. The review guard needs the sale's Customer_id and Status ('open', 'completed', 'voided') and the items of the sale (Transaction_id, Product_id).

What the built triggers already do. None of the five touches carts, addresses, phones, reviews or notifications, so keeping those correct is your job in the app. They do change `PRODUCT.Stock_quantity` (sale completed, supplier order received), low-stock alerts, and a customer's points and tier. Do not rebuild any of that.

7. Build order

Unlike the Part 2 guide, the SQL does not come first: the walking skeleton (browse, add to cart) was due Oct 19, so catalog and cart come first. `TR_REVIEW_Guard` is small and comes next because the lead needs your `triggers.sql` and `test.sql` by Oct 21. The order is a suggestion; the dates come from the plan.

Phase 0: Sync with the lead (before writing code)

- Ask what is already in main for the catalog and the cart, so you do not rebuild it.
- Get the seed script and the SQL files, and confirm the helper names in `lib/`.
- Confirm the `VW_CART` columns and the effective price rule with Part 2 (see the list at the end of this section).

Phase 1: Catalog and cart (the walking skeleton)

- **SQL** in `db/storefront/`: the catalog list query and the product detail query. List: search by name or SKU, filter by category, condition and price range, and a sort. Only `Is_active = 1`. Return the effective price: Sell_price minus the larger saving of the product discount and the category discount, each only if active and in date.
- Detail adds the average rating and the reviews.
- `api/products/` (list, detail), with a Jest test that calls the handler directly.
- `api/cart/`: add, update quantity and remove, reading through `VW_CART`. Customer id from the token. Every quantity change sets `Added_at` in the same UPDATE. A quantity of 0 means delete the row, because the table requires Quantity above 0.
- Screens: catalog with filters, and the cart page, which links to `/checkout`. Each handles loading, empty and error states.

Phase 2: TR_REVIEW_Guard and its tests (due to the lead Oct 21)

Trigger	Table and event	Rule
`TR_REVIEW_Guard`	REVIEW, AFTER INSERT / UPDATE	When a review has a Transaction_id, that sale must belong to the same customer, must be 'completed', and must contain the reviewed product. A review with no Transaction_id is always allowed.

- Put it in `db/storefront/triggers.sql`, starting with `USE pos_store;` then `GO`. Use `CREATE OR ALTER TRIGGER`, first in its batch, and model it on the built triggers.
- Handle multi-row statements through `inserted`. Pick a `THROW` number that is not 50001 or 50002 (ask the lead which are free).
- A NULL Transaction_id must pass. That is the normal case, and `ON DELETE SET NULL` produces it when a linked sale is deleted.
- Do not edit the five existing triggers.

Tests in `db/storefront/test.sql`. Each failing case runs in its own transaction that is rolled back, because a failing statement leaves the transaction unusable.

Required test	Expected result
Review linked to another customer's transaction	Rejected by TR_REVIEW_Guard
Review linked to a transaction that is not completed	Rejected by TR_REVIEW_Guard
Review linked to a product that is not in that sale	Rejected by TR_REVIEW_Guard

Recommended extras: a valid linked review is accepted; a NULL Transaction_id is accepted; a multi-row insert with one bad row is rejected as a whole. Test data: use the seed script's completed sales for the completed cases and your own open sale for the not-completed case. If you build a completed sale yourself once Part 2's payment triggers exist, follow the checkout order: items, totals, payments equal to Total, then update to 'completed'. When done, send `triggers.sql` and `test.sql` to the lead. If Oct 21 is at risk, tell the lead: the trigger is first on the cut list and can fall back to an app check in the same database transaction as the write.

Phase 3: Product page and reviews

- `api/reviews/`: list reviews for a product and post a review (posting is cut-first). Customer id from the token, Rating 1 to 5.
- To link a review to a purchase, offer only the customer's completed sales that contain the product; the guard rejects anything else.
- Screen: product page with the average rating, the reviews and a review form. Jest test.

Phase 4: Account

- `api/account/`: profile, addresses, phones and notifications. Customer id from the token.
- Addresses: add, edit, delete and set the default, inside one database transaction so the one-default rule never breaks.
- Notification preferences: one checkbox each for `Notify_cart` and `Notify_offers`. Phones are cut-first. Notifications list: show them and set `Read_at` when read.

- Screens: account (profile, addresses, phones, notification preferences) and notifications. Jest tests.

Phase 5: Notification job

- `api/jobs/notifications.js` is a protected endpoint. It creates cart reminders and marks them sent (`Sent_at`). In-site only.
- **Cart reminder rule:** the customer's cart has been idle for more than 24 hours, and no cart reminder was created for them in the last 3 days. Skip customers whose `Notify_cart` is 0. **Offer notifications** (cut first) skip customers whose `Notify_offers` is 0.
- Running it twice must not send duplicates: check the 3-day rule in the same statement or transaction that inserts. Keep the handler independent of how it is called, so Cron and a manual "run now" both work. Jest test, including a customer with a flag off.

Phase 6: Finish

- On Oct 22, check the full chain through the app: after a Part 2 checkout the cart is empty and the catalog shows the new stock. Then open a pull request from `part-1-storefront` to the lead.

Decide and write down before coding (suggestions)

- Is a 'fixed' discount applied per unit or per line, and how is it rounded? Match Part 2's `sp_Checkout` so catalog and checkout agree.
- What does "cart idle" mean: the newest Added_at in the customer's cart is older than 24 hours.
- What happens to the visitor's browser cart when they log in (merge it into CART_ITEM or drop it)? Login belongs to the lead, so agree it with them.
- Do out-of-stock products show (with a label) or hide? The brief only says to hide inactive ones. Does the cart cap quantity at current stock? Checkout rejects too much anyway.
- Can a customer review a product more than once? The schema document does not say. Also settle the reminder wording and the catalog's page size and sort options.

8. Schedule, pacing and cut order

Date	Plan milestone	Suggested pacing for Part 1
Oct 17 to 19	Foundation (17), schema freeze (18), walking skeleton (19): **all passed**	Phase 0: find out what exists
Oct 20 (Day 5, today)	Parts 1 to 4 build	Phases 0 and 1: catalog and cart
Oct 21 (Day 6)	**triggers.sql and test.sql due to the lead** (all parts except Part 2)	Phase 2, then start Phase 3
Oct 22 (Day 7)	Last build day; full chain runs through the app	Finish Phases 3 and 4, then Phase 5
Oct 23 (Day 8)	**Feature freeze**	Phase 6; no new features
Oct 23 to 24	Test and bug bash	Fix bugs found by the team
Oct 25	Submission	One .bak or .sql file (schema, then triggers)

The pacing column is a suggestion built from the plan dates; the middle column comes from the project plan. It is tight, so tell the lead early if you slip. The schema freeze has passed: do not change tables without the lead's approval.

If you fall behind, cut in this order

1. `TR_REVIEW_Guard` (fall back to an app check), offer notifications, phone management and posting reviews. The phones section of the account page goes with phone management.
2. The notification job becomes a manual "run now" endpoint (the shared cut order's last step).

If a teammate misses their must-haves, the lead takes over in this order: Part 2 (checkout), Part 4's alert-to-restock flow, Part 3's refund, then Part 1's catalog. The never-cut list belongs to other parts.

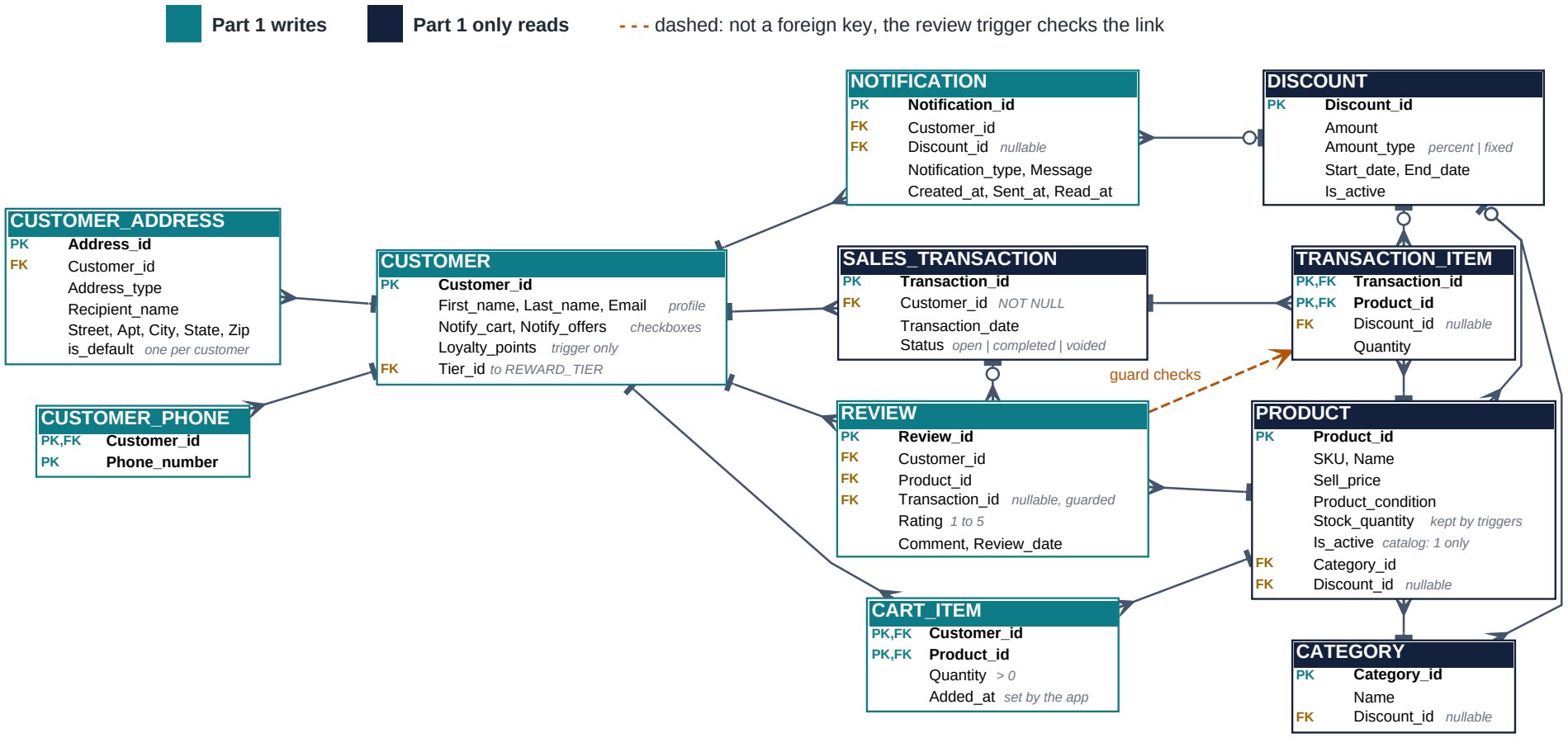
9. Definition of done

[ ]	`TR_REVIEW_Guard` is built, named correctly, set-based, and passes its tests in `db/storefront/test.sql` (each failing case in its own rolled-back transaction). `triggers.sql` and `test.sql` are with the lead.
[ ]	Each endpoint (products list and detail, reviews, cart, account, notification job) has a Jest test that calls the handler directly.
[ ]	Each screen (catalog, product page, cart, account, notifications) handles loading, empty and error states. The catalog shows only active products, and its effective price matches Part 2's checkout.
[ ]	Cart changes set `Added_at`, and the cart page only links to `/checkout`. Exactly one default address is kept per customer.
[ ]	The job skips customers with the matching flag off, sends no duplicates, and works from Cron or a manual run.
[ ]	Nothing writes `Loyalty_points` or `Tier_id`, no secrets are committed, and customer ids come from the token.
[ ]	The pull request is open against the lead.

10. Questions to confirm with the lead

- What is already in main for the catalog and cart (due Oct 19), and are the seed script, `lib/` and login merged?
- The final `VW_CART` columns, and whether they were agreed with Part 2.
- How the notification job is started and how its endpoint is protected.
- Which THROW numbers are free, and whether the Oct 21 deadline for `triggers.sql` and `test.sql` still stands.
- How the visitor's browser cart is handled at login, the exact `.env.local` variable names, and how the Azure database login will be shared.

Figure 1. The tables behind the storefront and how they connect



Reading the lines. A bar means one parent row, a crow's foot means many child rows, and a circle means optional (the foreign key can be NULL). Solid lines are foreign keys. The dashed line is not a foreign key: `TR_REVIEW_Guard` compares REVIEW.Transaction_id with the sale (same customer, status completed) and with its items (the product is in the sale).

CUSTOMER is teal because you edit the profile fields and the two Notify_* flags. `Loyalty_points` and `Tier_id` are set by triggers and are never written by the app. Tier_id points to REWARD_TIER, which is not drawn.

Cart and notifications. The notification job reads CART_ITEM (Added_at), the customer's flags and earlier NOTIFICATION rows, and inserts new NOTIFICATION rows. At checkout Part 2 reads the cart through VW_CART, copies the rows into TRANSACTION_ITEM and deletes them; there is no foreign key between the two tables.

Effective price is not stored. Compute it from PRODUCT, its own DISCOUNT and the DISCOUNT of its CATEGORY, using only discounts that are active and in date, and take the larger saving.

Not drawn. EMPLOYEE, REWARD_PROGRAM, REWARD_TIER, SALE_EVENT, LOYALTY_TRANSACTION, PAYMENT and the self-reference CUSTOMER.Referred_by_id. Part 1 does not write them.